Inteligencia Artificial Generativa Aplicada en Biomedicina

Clase 5: Retrieval-Augmented Generation (RAG) & Model Context Protocol (MCP)

Autor: Paulo Veiga / Marcos Sanchez Sorondo

Última Modificación: Abril, 2026

UNIVERSIDAD AUSTRAL

Agenda

1. **RAG: Fundamentos**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking y Metadatos**
   Fragmentación de documentos, estrategias de chunking y metadatos esenciales.
5. **RAG: Evaluación y Seguridad**
   Riesgos y evaluacion
6. **MCP: Fundamentos**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

RAG: FUNDAMENTOS

¿Qué es RAG?

> **RAG combina lo mejor de dos mundos: el razonamiento de los LLMs con el conocimiento actualizado de bases de datos externas.**

Recuperador

Busca información relevante en fuentes externas

LLM

Razona y genera la respuesta con el contexto recuperado

Base de conocimiento

Documentos, artículos y guías clínicas actualizadas

RAG: FUNDAMENTOS

Los cuatro pasos de RAG

RETRIEVE · Recuperar

Encontrar documentos relevantes para la consulta del usuario dentro de la base de conocimiento del sistema.

AUGMENT · Aumentar

Insertar los documentos recuperados en el prompt como contexto adicional que el modelo puede utilizar.

GENERATE · Generar

El LLM responde usando únicamente el contexto proporcionado, fundamentando y citando su respuesta.

Esta arquitectura crea un ciclo virtuoso donde la recuperación informa la generación, y el sistema aprende a mejorar ambos procesos simultáneamente.

¿Cuándo necesitás RAG?

Datos actualizados

Los datos de entrenamiento tienen fecha de corte. RAG recupera datos actuales desde fuentes en tiempo real.

Datos propietarios

Tus documentos internos y APIs nunca estuvieron en el entrenamiento. RAG conecta el LLM a tu base de conocimiento privada.

Precisión y citas

El modelo puede alucinar respuestas incorrectas. RAG restringe la generación a hechos recuperados con citas de fuentes.

Costo

El fine-tuning es caro y lento. Con RAG solo actualizás tu repositorio de documentos — sin reentrenamiento.

Auditabilidad

No podés rastrear por qué el modelo dijo algo. RAG enlaza cada respuesta a documentos fuente específicos.

RAG: FUNDAMENTOS

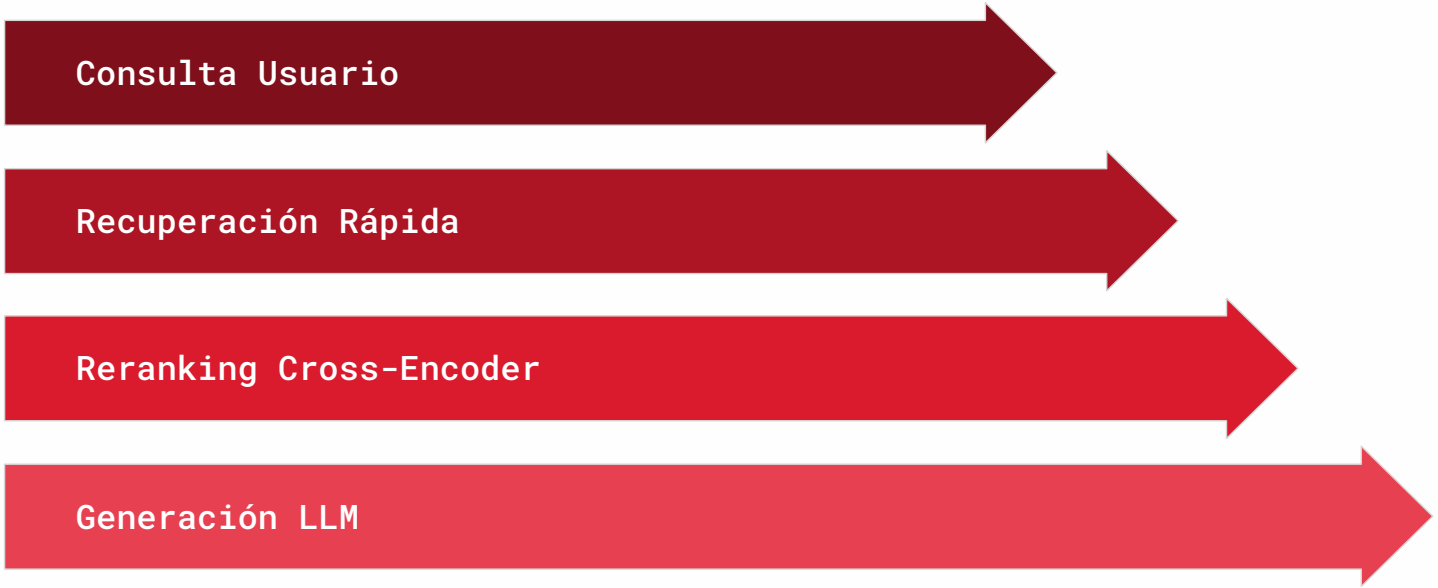
Ejemplo Básico de RAG

Tutorial interactivo: implementación paso a paso de un pipeline RAG con LangChain

Haz clic aquí

RAG: FUNDAMENTOS

Pipeline de RAG en producción



Procesamiento de documentos

Chunking, extracción de metadatos y filtrado de calidad.

Recuperación en dos etapas

Primera pasada rápida seguida de reranking preciso.

Ingeniería de contexto

Diseño de prompts, formato de citas y manejo de contexto insuficiente.

Evaluación continua

Métricas de retrieval (Recall@k, NDCG) y generación (faithfulness).

Observabilidad

Monitoreo de calidad, latencia y coste por consulta en tiempo real.

Agenda

1. **RAG: Fundamentos ✓**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking y Metadatos**
   Fragmentación de documentos, estrategias de chunking y metadatos esenciales.
5. **RAG: Evaluación, Seguridad y Patrones Avanzados**
   Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.
6. **MCP: Fundamentos**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

Fuentes de conocimiento

Documentos no estructurados

PDFs, artículos científicos, guías clínicas, informes. La fuente más común en biomedicina.

Bases de datos estructuradas

Registros clínicos, bases de datos relacionales, ontologías médicas (SNOMED, ICD-10).

APIs y fuentes en tiempo real

Datos de mercado, feeds de noticias, resultados de búsqueda web, APIs REST.

Código y datos técnicos

Repositorios de código, logs de sistemas, datos de sensores, registros de laboratorio.

La calidad y cobertura de las fuentes determina directamente la calidad de las respuestas RAG. *Garbage in, garbage out*.

RAG: BÚSQUEDA Y RECUPERACIÓN

Estructuras de Bases de Datos

Índice Invertido

BM25 / Keywords

- búsqueda por palabras exactas
- alta velocidad en consultas exactas
- ideal para terminología especializada (ICD-10, SKUs, términos legales)

Mejor para: Búsqueda exacta y filtrado por keywords

Vendors: Elasticsearch · OpenSearch · Apache Solr · Typesense

Base de Datos Vectorial

Embeddings semánticos

- captura significado y contexto semántico
- búsqueda por similitud entre conceptos
- soporta búsquedas multilenguaje

Mejor para: Búsqueda semántica y lenguaje natural

Vendors: Pinecone · Weaviate · Qdrant · Chroma · pgvector

Búsqueda Híbrida

BM25 + Vectorial

- combina precisión léxica con comprensión semántica
- reranking para fusionar resultados
- estándar en sistemas RAG de producción

Mejor para: Sistemas RAG en producción ⭐

Vendors: Elasticsearch · Weaviate · Azure AI Search · MongoDB Atlas

La elección entre ambas estrategias —o su combinación— depende del tipo de consulta y la naturaleza del corpus documental disponible.

RAG: BÚSQUEDA Y RECUPERACIÓN

Ejemplo Básico de RAG

Tutorial interactivo: implementación paso a paso de un pipeline RAG con LangChain

Haz clic aquí

RAG: BÚSQUEDA Y RECUPERACIÓN

¿Cómo funciona un Índice Invertido?

Ingesta de documentos

Se reciben los documentos de texto. Ej: Doc1: "el gato come pescado", Doc2: "el perro come carne", Doc3: "el gato y el perro"

Tokenización y normalización

Cada documento se divide en tokens (palabras). Se aplica lowercase, eliminación de stopwords ("el", "y") y stemming. Resultado: Doc1→[gato, come, pescado], Doc2→[perro, come, carne], Doc3→[gato, perro]

Construcción del índice

Se crea un mapa invertido: cada término apunta a los documentos que lo contienen. Ej: "gato" → [Doc1, Doc3] · "come" → [Doc1, Doc2] · "perro" → [Doc2, Doc3]

Consulta (Query)

El usuario busca "gato perro". El índice devuelve la intersección: Doc3 (contiene ambos). Ranking por frecuencia/BM25.

BM25 extiende este modelo con ponderación por frecuencia de término (TF) e infrecuencia en el corpus (IDF), mejorando el ranking de resultados.

RAG: BÚSQUEDA Y RECUPERACIÓN

TF-IDF: Term Frequency × Inverse Document Frequency

> 💡 **Idea: Un término es importante si aparece mucho en un documento... pero no si aparece en todos los documentos.**

TF (Term Frequency)

TF responde: ¿Qué tan frecuente es este término en este documento?

```
TF(t, d) = count(t in d) / total_terms(d)
```

Qué tan frecuente es el término en el documento. Ej: "gato" aparece 3 veces en un doc de 100 palabras → TF = 0.03

IDF (Inverse Document Frequency)

IDF responde: ¿Qué tan raro es este término en todo el corpus?

```
IDF(t) = log(N / df(t))
```

Penaliza términos que aparecen en muchos documentos (son poco informativos). Ej: "el", "de" → IDF ≈ 0. "leucemia" → IDF alto.

t = un término (palabra o token) específico. Es el término cuya importancia queremos medir. Ejemplo: t = "leucemia", t = "gato", t = "el".

d = un documento específico del corpus. Es el documento para el cual estamos calculando el score. Ejemplo: "Doc 1: el gato come pescado fresco" → d = Doc 1.

N = número total de documentos en el corpus.

Ejemplo: si tenemos 10.000 artículos médicos, **N = 10.000**.

df(t) = número de documentos en los que aparece el término *t*.

Ejemplo: "leucemia" aparece en 200 documentos → **df("leucemia") = 200**.

Score final:

```
TF-IDF(t, d) = TF(t, d) × IDF(t)
```

⚠️ Limitación: TF-IDF no satura — un término que aparece 100 veces vale 100x más que uno que aparece 1 vez. → BM25 resuelve esto.

RAG: BÚSQUEDA Y RECUPERACIÓN

TF-IDF en acción: Ejemplo visual

El corpus (3 documentos)

Doc 1

el gato come **pescado fresco**

5 tokens

Doc 2

el perro come **carne** y hueso

6 tokens

Doc 3

el gato y el perro son **amigos**

7 tokens

Índice invertido con scores TF-IDF

Término	En docs	TF (Doc1)	IDF	TF-IDF
el	Doc1, Doc2, Doc3	2/5=0.40	log(3/3)=0.00	**0.00** (stopword)
gato	Doc1, Doc3	1/5=0.20	log(3/2)=0.18	**0.04**
come	Doc1, Doc2	1/5=0.20	log(3/2)=0.18	**0.04**
pescado	Doc1	1/5=0.20	log(3/1)=0.48	**0.10** ⭐
fresco	Doc1	1/5=0.20	log(3/1)=0.48	**0.10** ⭐

¿Por qué funciona?

"el" aparece en los 3 docs → IDF = log(3/3) = 0 → score 0. Las stopwords se eliminan solas.

"gato" aparece en 2/3 docs → IDF bajo → score bajo. Es común en el corpus.

"pescado" y "fresco" solo en Doc1 → IDF alto → score alto. Son los términos más discriminativos.

💡 El índice invertido es como el índice de un libro: vas directo al término y te dice en qué documentos aparece y con qué relevancia.

RAG: BÚSQUEDA Y RECUPERACIÓN

TF-IDF: De la búsqueda al ranking de documentos

Query del usuario

"¿Qué documentos hablan de pescado fresco?" → Se tokeniza igual que los docs: [pescado, fresco]

Lookup en el índice

Se buscan las posting lists: "pescado" → [Doc1] "fresco" → [Doc1] Intersección: Doc1 ✓

Cálculo de scores TF-IDF

Doc1: TF-IDF("pescado") = 0.22 TF-IDF("fresco") = 0.22 Score total = 0.44 🥇

Ranking y entrega al LLM

Se ordenan docs por score descendente. Doc1 (0.44) → top resultado. Se inyecta como contexto en el prompt RAG.

Query: 'pescado fresco' → Ranking de resultados

Score	Contenido	Cálculo	Documento	Entregado al LLM
0.44 ✅	"el gato come pescado fresco"	TF("pescado",Doc1) = 1/5 = 0.20 IDF("pescado") = log(3/1) = 1.10 TF("fresco",Doc1) = 1/5 = 0.20 IDF("fresco") = log(3/1) = 1.10 Score = (0.20×1.10) + (0.20×1.10) = **0.44** ✅	Doc 1	✅ Sí (top resultado)
0.00 ❌	"el perro come carne y hueso"	TF("pescado",Doc2) = 0 → 0.00 TF("fresco",Doc2) = 0 → 0.00 Score = **0.00** ❌	Doc 2	❌ No (sin match)
0.00 ❌	"el gato y el perro son amigos"	TF("pescado",Doc3) = 0 → 0.00 TF("fresco",Doc3) = 0 → 0.00 Score = **0.00** ❌	Doc 3	❌ No (sin match)

Solo Doc1 contiene los términos 'pescado' y 'fresco' → score 0.44. Los demás documentos no matchean ningún término de la query → score 0.00 → no se entregan al LLM.

💡 El índice TF-IDF actúa como el 'motor de búsqueda' del pipeline RAG: convierte millones de documentos en una lista ordenada de candidatos relevantes en milisegundos, antes de que el LLM genere la respuesta.

RAG: BÚSQUEDA Y RECUPERACIÓN

Ejemplo Indice Invertido

Haz clic aquí

RAG: BÚSQUEDA Y RECUPERACIÓN

¿Cómo funciona la búsqueda vectorial?

1 Conversión a vectores

Transforma el texto en representaciones numéricas multidimensionales mediante modelos de embeddings entrenados para capturar semántica.

2 Cálculo de similitud

Mide la distancia o similitud semántica entre vectores usando métricas como similitud coseno o distancia euclidiana.

3 Recuperación inteligente

Encuentra documentos relevantes aunque no contengan las palabras exactas de la consulta, entendiendo la intención subyacente.

Permite búsquedas más flexibles y contextuales, entendiendo la intención real detrás de las consultas del usuario.

Embeddings: Convirtiendo texto en vectores

> 💡 Idea: Las palabras con significado similar deben estar cerca en el espacio vectorial. 'gato' y 'felino' deben ser vecinos. 'gato' y 'automóvil' deben estar lejos.

¿Qué es un embedding?

Un embedding convierte texto en un punto en un espacio de N dimensiones.

```
"gato"  → [0.82, -0.31, 0.54, 0.12, ...]  (384 dims)
"felino" → [0.79, -0.28, 0.51, 0.15, ...]  (384 dims)
"perro"  → [0.71, -0.22, 0.48, 0.09, ...]  (384 dims)
"auto"   → [-0.12, 0.65, -0.33, 0.87, ...] (384 dims)
```

Modelos populares: text-embedding-3-small (OpenAI), all-MiniLM-L6-v2 (open source), embed-multilingual-v3 (Cohere)

¿Cómo se entrena?

→ Entrenamiento contrastivo

El modelo aprende que frases con significado similar deben producir vectores cercanos. Se entrena con millones de pares (pregunta, respuesta relevante).

→ Dimensionalidad

Los vectores tienen entre 384 y 3072 dimensiones. Más dimensiones = más capacidad semántica, pero más costo de almacenamiento y cómputo.

→ Multilingüe

Los modelos multilingües mapean "cat" (inglés) y "gato" (español) a vectores cercanos — la búsqueda funciona entre idiomas sin traducción.

💡 El embedding captura semántica, no sintaxis. 'No puedo iniciar sesión' y 'olvidé mi contraseña' producen vectores muy cercanos aunque no compartan ninguna palabra.

RAG: BÚSQUEDA Y RECUPERACIÓN

Similitud Coseno: Midiendo cercanía semántica

> 💡 Idea: No nos importa el tamaño del vector, sino su dirección. Dos textos son similares si apuntan en la misma dirección en el espacio vectorial.

La fórmula

Similitud coseno responde: ¿Qué tan parecida es la dirección de dos vectores?

$$\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$$

Where:
- A · B = producto punto (suma de productos elemento a elemento)
- ||A|| = magnitud del vector A (raíz de suma de cuadrados)
- Resultado: entre -1 (opuestos) y 1 (idénticos)

Par de términos	Similitud coseno	Interpretación
"gato" vs "felino"	0.91	Muy similares ✅
"gato" vs "perro"	0.76	Relacionados
"gato" vs "auto"	0.12	Sin relación ❌
Query vs Doc1	0.87	Alta relevancia ⭐

Intuición geométrica

→ Ángulo, no distancia

cos(0°) = 1 → idénticos. cos(90°) = 0 → sin relación. cos(180°) = -1 → opuestos. Usamos el ángulo entre vectores, no su distancia absoluta.

→ Invariante a la longitud

"gato" y "El gato doméstico es un felino" producen vectores de distinto tamaño pero dirección similar → similitud alta. Esto es clave para comparar chunks de distinto largo.

→ Alternativas: distancia euclidiana y producto punto

Distancia euclidiana: más intuitiva pero sensible a la magnitud. Producto punto: más rápido, usado en ANN (Approximate Nearest Neighbors) para escala.

> 💡 En producción, la búsqueda exacta de vecinos más cercanos es O(n) — demasiado lenta para millones de vectores. Se usan índices ANN (HNSW, IVF) que sacrifican un poco de precisión por velocidad sub-lineal.

RAG: BÚSQUEDA Y RECUPERACIÓN

Búsqueda Vectorial: De la query al ranking de documentos

Query del usuario

"¿Qué animales comen pescado?" → se convierte en embedding: [0.81, -0.29, 0.52, ...]

Búsqueda ANN

Se buscan los K vectores más cercanos en la base de datos vectorial usando índice HNSW o IVF. Sub-lineal en tiempo.

Score de similitud coseno

Se calcula cos(θ) entre el vector de la query y cada candidato recuperado.

Ranking y entrega al LLM

Los documentos se ordenan por similitud descendente. Los top-K se inyectan como contexto en el prompt RAG.

Query: '¿Qué animales comen pescado?' → Ranking vectorial

Score coseno	Contenido	Documento	Entregado al LLM
0.91	"el gato come pescado fresco"	Doc 1	✅ Sí — alta similitud semántica
0.43	"el gato y el perro son amigos"	Doc 3	❌ No — menciona animales pero no alimentación
0.21	"el perro come carne y hueso"	Doc 2	❌ No — alimentación pero no pescado

A diferencia de TF-IDF, Doc1 es recuperado aunque la query no contenga exactamente 'gato' ni 'pescado' — la similitud semántica captura la intención.

💡 Ventaja clave sobre TF-IDF: 'infarto de miocardio' y 'ataque al corazón' tienen similitud coseno ~0.89 → el sistema vectorial los conecta. TF-IDF los trataría como documentos completamente distintos.

RAG: BÚSQUEDA Y RECUPERACIÓN

Ejemplo Busqueda Semantica

Haz clic aquí

RAG: BÚSQUEDA Y RECUPERACIÓN

Estrategias de búsqueda: Léxica vs Semántica vs Híbrida

Búsqueda Léxica

BM25 / Keywords

- ✓ Rápida, precisa en términos exactos, sin GPU
- ⊗ No reconoce sinónimos ni abstracciones
- Ideal: Códigos ICD-10, terminología legal
- Coste: ~$0.0001/query

Búsqueda Semántica

Embeddings / Vectores

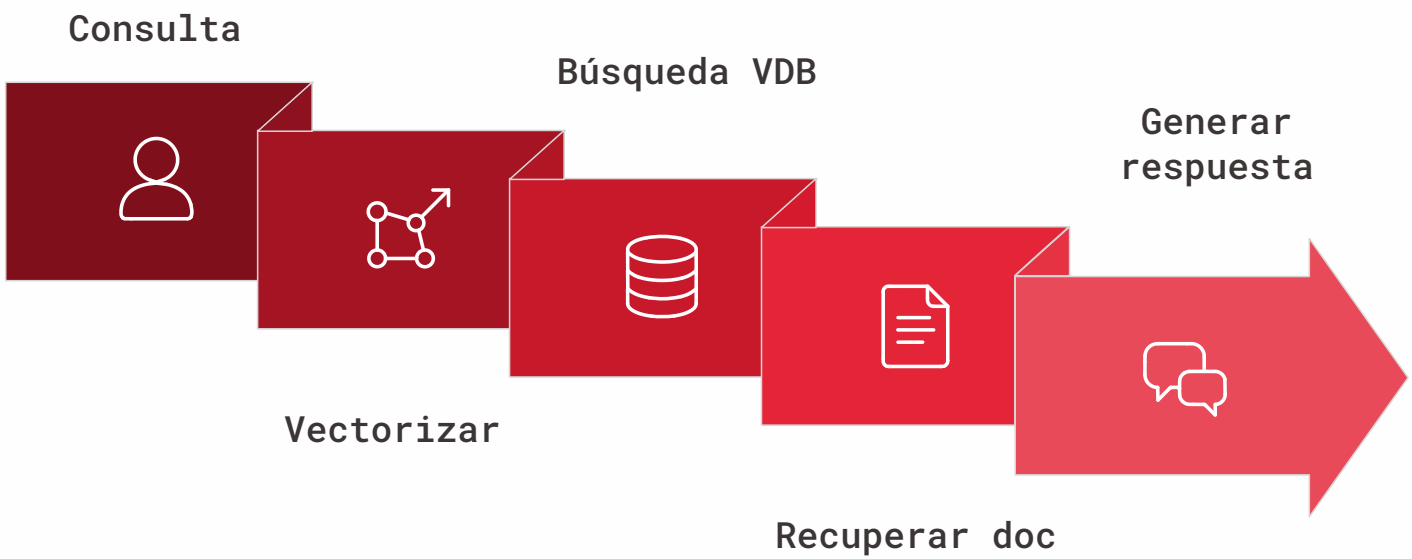
- ✓ Sinónimos, multilenguaje, relaciones conceptuales
- ⊗ Más lenta (100-500ms), requiere GPU
- Ideal: Papers académicos, FAQs, help desk
- Coste: ~$0.001-0.01/query

Búsqueda Híbrida

Léxica + Semántica

- ✓ Mejor cobertura, 85-92% relevancia
- ⊗ Mayor complejidad, requiere tuning de pesos
- Ideal: Producción general, consultas biomedicina
- Coste: ~$0.002-0.015/query

Flujo completo de RAG



El sistema completo trabaja en milisegundos para proporcionar respuestas precisas y fundamentadas en datos verificables, sin importar el tamaño del corpus.

RAG: BÚSQUEDA Y RECUPERACIÓN

El desafío del tamaño del contexto

Límite de tokens

Los LLMs tienen un límite máximo de tokens que pueden procesar simultáneamente en una consulta. Superar este límite produce errores o truncado de información.

Documentos extensos

Documentos largos o múltiples fuentes de información pueden exceder fácilmente ese límite técnico, especialmente en corpus biomédicos.

Solución RAG

RAG selecciona inteligentemente solo la información más relevante para incluir en el contexto, optimizando el uso de la ventana disponible.

Agenda

1. **RAG: Fundamentos ✓**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación ✓**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking y Metadatos**
   Fragmentación de documentos, estrategias de chunking y metadatos esenciales.
5. **RAG: Evaluación, Seguridad y Patrones Avanzados**
   Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.
6. **MCP: Fundamentos**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

RAG: RERANKING Y PRECISIÓN

¿Qué es el Reranking?

> 💡 La recuperación rápida encuentra candidatos pero los rankea mal. El reranking filtra el ruido y se queda solo con los documentos verdaderamente relevantes.

Etapa 1: Recuperación rápida (Recall)

Objetivo: No perder documentos relevantes – red amplia

- → **Método:** BM25 léxico + búsqueda vectorial en paralelo
- → **Resultado:** top-50 candidatos recuperados rápidamente
- → **Trade-off:** rápido pero incluye ruido e irrelevantes

Etapa 2: Reranking (Precision)

Objetivo: Quedarse solo con los mejores – filtrar el ruido

- → **Método:** Cross-encoder analiza cada candidato en profundidad
- → **Resultado:** top-3 a top-5 documentos de alta precisión
- → **Trade-off:** más lento pero mucho más preciso en relevancia

100K+	50	5
docs		
Corpus total (búsqueda rápida lo maneja)	Candidatos tras Stage 1	Resultado final tras Reranking ⭐

> 💡 ¿Por qué funciona? Los retrievers rápidos son buenos encontrando candidatos pero malos rankeándolos. Los cross-encoders son excelentes rankeando pero demasiado lentos para miles de documentos. Combinados: velocidad + precisión.

RAG: RERANKING Y PRECISIÓN

Pipeline completo: Retrieval + RRF + Reranking

El pipeline combina recuperación léxica y semántica en paralelo, fusiona los resultados con RRF, y aplica un cross-encoder para quedarse con los top-5 más precisos.

Recuperación en paralelo

BM25 léxico + búsqueda vectorial corren simultáneamente. Cada uno retorna top-50 candidatos.

Reciprocal Rank Fusion (RRF)

Fusiona los dos rankings en uno solo. RRF(d) = Σ 1/(k + rank(d)). Penaliza documentos que aparecen tarde en ambas listas.

Cross-Encoder Reranking

Analiza cada par (query, documento) en profundidad. Mucho más preciso que bi-encoders. Reduce de top-50 a top-5.

Contexto al LLM

Los top-5 documentos de alta precisión se inyectan en el prompt. Mejor contexto → mejor respuesta.

¿Qué es un Cross-Encoder?

A cross-encoder recibe la query Y el documento juntos como input:

```
input: "[CLS] query [SEP] documento [SEP]"
output: score de relevancia (0 a 1)
```

- Analiza la interacción entre query y documento
- Mucho más preciso que similitud coseno
- Modelos: Cohere Rerank, BGE-Reranker, ms-marco-MiniLM
- Limitación: O(n) — no escala a millones de docs

RRF: Reciprocal Rank Fusion

RRF(d) = Σ 1 / (k + rank_i(d)) [k=60 por defecto]

Documento	Rank BM25	Rank Vectorial	RRF Score
Doc A	1	3	0.032
Doc B	5	1	0.032
Doc C	2	8	0.028

RRF es robusto: no requiere normalizar scores de distintas fuentes.

💡 Tips de producción: mantener el pool de candidatos en 20-100 docs y el top_k final en 3-5. Correr BM25 y vectorial en paralelo. Loggear latencia y costo por consulta.

RAG: RERANKING Y PRECISIÓN

🤔 ¿Por qué concatenar query + documento en el ranking con embeddings?

> Un bi-encoder embeddea query y documento por separado y compara vectores. Un cross-encoder los lee juntos y entiende la relación entre ambos. Esa diferencia lo cambia todo.

Bi-encoder: embeddings separados

Así funciona la búsqueda vectorial clásica.

embed(query)	vector A → [0.81, -0.29, 0.52, ...]
embed(documento)	vector B → [0.79, -0.28, 0.51, ...]
score	cos(A, B) = 0.91

- **Escalable — O(1) por doc en query time** — Los embeddings de documentos se pre-computan y almacenan. Solo se embeddea la query en tiempo real. Ideal para millones de docs.
- **Pierde el contexto conjunto** — El embedding de la palabra banco es el mismo sin importar si la query habla de dinero o de un río. No ve la interacción entre ambos textos.
- **Precisión limitada en reranking** — Bueno para recuperar candidatos, pero no suficientemente preciso para decidir cuál de los top-50 es realmente el más relevante.

Cross-encoder: query + doc juntos

Así funciona el reranker — lee ambos a la vez.

Input al modelo:	query + separador + documento (texto completo)
Procesamiento:	Transformer atiende cada token de la query contra cada token del doc
Output:	score de relevancia entre 0 y 1

- **Ve la interacción completa** — El modelo puede detectar que pescado en el documento responde directamente a la pregunta sobre alimentación de animales en la query.
- **Mucho más preciso** — Distingue matices imposibles para el coseno: negaciones, contexto, ambigüedad léxica. Modelos: Cohere Rerank, BGE-Reranker, ms-marco-MiniLM.
- **No escala solo — O(n) por query** — Requiere una inferencia completa por cada par. Con 100K docs sería inviable. Por eso se usa solo sobre los top-50 candidatos del Stage 1.

💡 La concatenación es la clave: al pasarle query y documento juntos al Transformer, el mecanismo de atención puede relacionar directamente cada palabra de la pregunta con cada palabra del documento. Eso es lo que el coseno entre vectores separados nunca puede hacer.

MCP: ARQUITECTURA Y USO

Ejemplo de Ranking

Haz clic aquí

Agenda

1. **RAG: Fundamentos ✓**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación ✓**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking y Metadatos**
   Fragmentación de documentos, estrategias de chunking y metadatos esenciales.
5. **RAG: Evaluación, Seguridad y Patrones Avanzados**
   Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.
6. **MCP: Fundamentos**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

RAG: CHUNKING, METADATOS Y RERANKING

Técnicas de chunking: Fragmentación de documentos

El chunking divide documentos extensos en fragmentos manejables. La tensión fundamental: fragmentos pequeños (100-200 tokens) mejoran la precisión pero pierden contexto; fragmentos grandes (1000+) mantienen contexto pero diluyen la señal de recuperación.

Tamaño fijo

Divide por cantidad de tokens con overlap configurable. Simple y predecible, pero puede cortar oraciones en puntos inapropiados.

Semántico

Respeta límites de párrafos y secciones. Ofrece mejor coherencia semántica aunque produce tamaño variable.

Estructura-aware

Usa la jerarquía del documento (headers, secciones). Ideal para documentos técnicos, APIs y guías clínicas.

Con metadatos

Enriquece fragmentos con fuente, timestamp, posición y autor. Mejora el filtrado previo a la búsqueda.

Metadatos y guía por tipo de documento

Metadatos esenciales

Siempre incluir:

- Identificador de fuente
- Timestamp de creación/actualización
- Posición del chunk dentro del documento

Condicional:

- Autor / departamento
- Tipo de documento y versión
- Jerarquía de sección, idioma, score de calidad

Tamaño recomendado por tipo

Tipo	Estrategia	Chars
Blog posts	Semántico	800-1000
Docs técnicos	Estructura	600-800
Contratos	Estructura	1000-1500
Papers	Estructura	1000-1200

Agenda

1 **RAG: Fundamentos ✓**

Qué es RAG, por qué lo necesitamos y arquitectura básica.

2 **RAG: Búsqueda y Recuperación ✓**

Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.

3 **RAG: Reranking y Precisión**

Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.

4 **RAG: Chunking y Metadatos**

Fragmentación de documentos, estrategias de chunking y metadatos esenciales.

5 **RAG: Evaluación y Seguridad**

Riesgos y evaluacion

6 **MCP: Fundamentos**

Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.

7 **MCP: Arquitectura y Uso**

Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.

8 **MCP: Diseño de Herramientas**

Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

Evaluación de RAG: Métricas clave

Evaluar retrieval y generación por separado es fundamental. **El 70% de los fallos de RAG provienen de la recuperación, no de la generación**, por lo que dedicar esfuerzo a medir el retrieval es prioritario.

Context Relevance

¿Se recuperaron los documentos correctos? Mide si el recuperador trae contexto verdaderamente útil para responder la consulta.

Answer Faithfulness

¿La respuesta está fundamentada en el contexto? Evalúa si el LLM se limita a lo que hay en los documentos recuperados.

Answer Relevance

¿La respuesta aborda la pregunta del usuario? Verifica que la respuesta generada sea pertinente y completa.

Riesgos y desafíos de seguridad en RAG

Acceso no autorizado

Riesgo de exponer información sensible si no se controla adecuadamente la recuperación de datos. En biomedicina, esto incluye historiales clínicos y datos genómicos.

Explotación de vulnerabilidades

Actores maliciosos pueden manipular consultas mediante prompt injection para extraer datos privados o confidenciales del sistema.

Necesidad de controles

Políticas estrictas de acceso, auditoría continua y validación de entradas y salidas son fundamentales en cualquier despliegue productivo.

Ejemplo real de riesgo

> "Un chatbot empresarial con RAG filtra accidentalmente datos financieros confidenciales al no validar correctamente los permisos de acceso del usuario."

1 El problema

Sistema de permisos mal configurado permite el acceso a documentos marcados como restringidos durante la fase de recuperación.

2 El impacto

Información sensible queda expuesta a usuarios no autorizados, con potenciales implicaciones legales y regulatorias.

3 La lección

Validar y filtrar **siempre** la información recuperada antes de incluirla en el contexto y antes de mostrar la respuesta final.

Buenas prácticas para mitigar riesgos

1

Control de acceso granular

Implementar permisos detallados a nivel de documento y usuario en todas las bases de datos del sistema RAG.

2

Encriptación y anonimización

Proteger datos sensibles mediante cifrado en reposo y en tránsito; anonimizar información personal identificable.

3

Monitoreo continuo

Auditoría en tiempo real de consultas y accesos, complementada con pruebas de penetración periódicas.

4

Validación de salidas

Filtrar y revisar respuestas generadas antes de entregarlas al usuario final, evitando fuga de información.

Limitaciones actuales y retos futuros

Escalabilidad

Manejar eficientemente grandes volúmenes de datos y consultas concurrentes sin degradar el rendimiento ni disparar los costes operativos.

Integración optimizada

Mejorar la coordinación entre los módulos de recuperación y generación para producir resultados más coherentes y menos fragmentados.

Sesgos y transparencia

Evitar sesgos introducidos en la recuperación y generación, garantizando la explicabilidad de las respuestas para contextos regulados.

A medida que RAG evoluciona, la comunidad de investigación trabaja activamente en resolver estos desafíos para hacer la tecnología más robusta y confiable en entornos críticos.

Resumen y conclusiones de RAG

Superación de limitaciones

RAG es fundamental para superar las restricciones de los LLMs tradicionales en cuanto a conocimiento estático y alucinaciones.

Combinación poderosa

Integra recuperación inteligente con generación precisa y contextualizada para respuestas verificables y citables.

Cuidado necesario

Requiere atención especial en el diseño del chunking, la evaluación continua y la seguridad de los datos.

Adopción creciente

Su uso se expande en múltiples industrias: biomedicina, legal, finanzas, y cualquier dominio con información dinámica.

RAG representa un avance significativo en IA generativa. Dominar búsqueda híbrida, reranking y evaluación será clave para aplicaciones de próxima generación.

Agenda

1 **RAG: Fundamentos** ✓

Qué es RAG, por qué lo necesitamos y arquitectura básica.

2 **RAG: Búsqueda y Recuperación** ✓

Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.

3 **RAG: Reranking y Precisión**

Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.

4 **RAG: Chunking y Metadatos**

Fragmentación de documentos, estrategias de chunking y metadatos esenciales.

5 **RAG: Evaluación, Seguridad y Patrones Avanzados** ✓

Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.

6 **MCP: Fundamentos**

Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.

7 **MCP: Arquitectura y Uso**

Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.

8 **MCP: Diseño de Herramientas**

Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

MCP: FUNDAMENTOS

Model Context Protocol (MCP)

El **Protocolo de Contexto del Modelo** representa un avance fundamental en la integración de sistemas de IA con fuentes de datos y herramientas externas. Exploraremos los desafíos que motivaron su desarrollo, su arquitectura técnica y su impacto en el ecosistema de desarrollo de aplicaciones inteligentes.

MCP: FUNDAMENTOS

Necesidad: Integración de IA con sistemas externos

Conocimiento estático

Los LLMs operan con conocimiento almacenado durante su entrenamiento, sin capacidad de acceder a información en tiempo real o sistemas de producción.

Conectores personalizados

Cada integración requería el desarrollo de conectores únicos, generando costes elevados y un mantenimiento poco escalable para equipos de ingeniería.

Silos de información

La falta de estandarización creaba fragmentación de datos y duplicación masiva de esfuerzos de desarrollo en toda la industria.

MCP: FUNDAMENTOS

Historia y necesidad del MCP

> **Noviembre 2024 — Anthropic lanza MCP como protocolo abierto para toda la comunidad de desarrollo de IA.**

Visión «USB-C para IA»

Lenguaje común que conecta cualquier modelo con múltiples fuentes sin necesidad de conectores propietarios.

Reducción de complejidad

De M×N integraciones (cada modelo con cada fuente) a M+N conexiones estandarizadas y reutilizables.

Aceleración del desarrollo

Capa de abstracción universal que facilita la construcción rápida de aplicaciones inteligentes y componibles.

¿Por qué no usar solo HTTP o GraphQL?

HTTP: Generalidad sin especialización

- Heterogeneidad de interfaces entre servicios
- Sin negociación de capacidades entre cliente y servidor
- Sin gestión de sesiones persistentes
- Sin propagación de contexto semántico

GraphQL: Consultas potentes, orquestación limitada

- Foco en fetch de datos, no en ejecución de acciones
- Sin orquestación dinámica de múltiples fuentes
- Sin estandarización de operaciones asíncronas

MCP: El estándar universal para IA

MCP es un protocolo **bidireccional y optimizado** para la comunicación entre agentes de IA y sistemas externos. Resuelve exactamente las limitaciones de HTTP y GraphQL en contextos de IA:

- Negociación de capacidades en tiempo de conexión
- Sesiones persistentes con estado compartido
- Propagación nativa de contexto semántico
- Orquestación dinámica de herramientas y fuentes

Agenda

1. **RAG: Fundamentos ✓**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación ✓**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking y Metadatos ✓**
   Fragmentación de documentos, estrategias de chunking y metadatos esenciales.
5. **RAG: Evaluación, Seguridad y Patrones Avanzados ✓**
   Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.
6. **MCP: Fundamentos ✓**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

Protocolo JSON-RPC: Comunicación MCP

Fundamentos de JSON-RPC 2.0

MCP utiliza **JSON-RPC 2.0** como capa de transporte, permitiendo mensajes ligeros y estructurados entre cliente y servidor.

Flujo de comunicación:

- Cliente envía solicitud JSON con método y parámetros
- Servidor procesa y responde con resultado o error estructurado
- Protocolo sin estado con soporte para batch requests

Ejemplo HTTP POST

```
{
 "jsonrpc": "2.0",
 "method": "tools/call",
 "params": {
 "name": "get_weather",
 "arguments": {
 "location": "Madrid"
 }
 },
 "id": 1
}

Response:
{
 "temperature": 22,
 "condition": "soleado"
}
```

De herramientas hardcodeadas a MCP

Herramienta Hardcodeada

- La lógica de la herramienta reside en el código del agente.
- El agente conoce las herramientas en tiempo de compilación.
- Requiere copiar y pegar código para reutilizar la herramienta.
- Necesidad de redeploy del agente para cualquier actualización.
- Protocolo de comunicación específico del framework.

Servidor MCP (Protocolo de Contexto del Modelo)

- La herramienta opera como un proceso separado en localhost.
- El agente consulta `tools/list` en tiempo de ejecución para descubrir herramientas.
- Cualquier cliente compatible con MCP puede conectarse y usarlo.
- Solo es necesario reiniciar el servidor MCP para aplicar actualizaciones.
- JSON-RPC estandarizado sobre HTTP para la comunicación.

MCP separa las herramientas de los agentes — un servidor MCP puede ser usado por cualquier cliente compatible: Claude, ChatGPT, Cursor o tu propio agente.

Ciclo de vida de una herramienta MCP

01

El servidor MCP arranca

Declara sus herramientas: nombre, descripción y esquema de parámetros.

02

El agente se conecta

Como cliente MCP, descubre las herramientas disponibles vía tools/list.

03

El usuario envía una consulta

El LLM analiza las descripciones de herramientas y decide cuál invocar.

04

El agente llama la herramienta

A través del protocolo MCP con parámetros estructurados.

05

El servidor MCP ejecuta

Procesa la solicitud y devuelve un resultado estructurado.

06

El LLM usa el resultado

Continúa el razonamiento o responde al usuario con la información obtenida.

Principios de diseño de herramientas MCP

Principio 1: Nombres claros y descriptivos

Convención: **[verbo]_[sustantivo]_[contexto]**

Ejemplos correctos: get_customer_by_email, search_products_by_category, calculate_shipping_cost_for_order

Evitar: process, fetch, do_thing

Principio 2: Descripciones exhaustivas

La descripción es lo más importante. Debe responder: ¿Qué hace? ¿Cuándo usarla? ¿Cuándo NO usarla? ¿Qué formato tienen entradas/salidas?

Principio 3: Esquemas de parámetros simples

Investigación muestra: 1-3 parámetros → 90%+ precisión | 4-6 parámetros → 75-85% | 7+ parámetros → 60-70%. Preferir múltiples herramientas simples sobre una herramienta compleja.

Principio 4: Formatos de respuesta consistentes

Usar un envelope estándar: { success, data, error, message }. El agente sabe qué esperar, facilita el manejo de errores y el control de éxito/fallo.

Cómo conectar un servidor MCP a Claude Desktop

Método 1 — Desktop Extensions (5 min)

1. Abre Claude Desktop → Configuración (⚙️) → Extensions
2. Busca el servidor deseado (ej. "Filesystem")
3. Haz clic en Instalar → Concede permisos
4. Inicia un nuevo chat y prueba: "Lista los archivos de mi carpeta Documentos"

Método 2 — Configuración manual (JSON)

Edita el archivo de configuración en:

- macOS: ~/Library/Application Support/Claude/claude_desktop_config.json
- Windows: %APPDATA%\Claude\claude_desktop_config.json

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/ruta/a/tu/proyecto"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "tu_token"
      }
    }
  }
}
```

Método 3 — Servidor MCP local (HTTP/SSE)

1. Asegurate de que el servidor esté corriendo en http://localhost:8002/mcp
2. Edita el archivo claude_desktop_config.json (mismas rutas que Método 2)
3. Agrega la entrada con transport "http":

```
{
  "mcpServers": {
    "mi-servidor-local": {
      "type": "http",
      "url": "http://localhost:8002/mcp"
    }
  }
}
```

Útil para servidores propios en desarrollo o herramientas internas de la organización.

4. Reinicia Claude Desktop y verificá que aparezca el ícono 🔌 en el chat.

⚠️ **Seguridad:** Usa servidores MCP de fuentes confiables. Los servidores que acceden a contenido externo pueden exponer a ataques de prompt injection. Verifica siempre el origen antes de instalar.

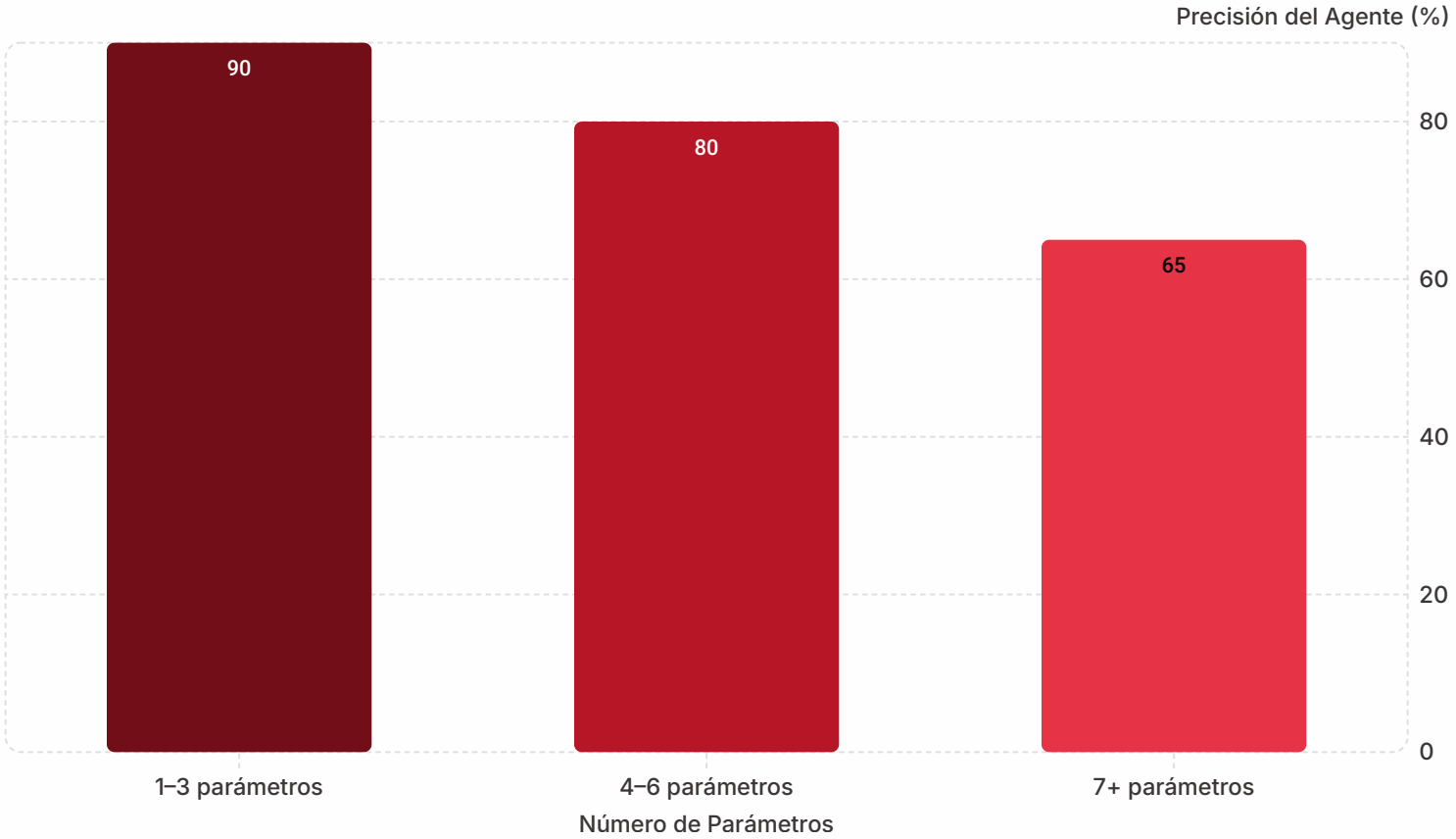
MCP: ARQUITECTURA Y USO

Ejemplo MCP Server

Haz clic aquí

MCP: ARQUITECTURA Y USO

Precisión del agente según complejidad de parámetros



Más parámetros = mayor carga cognitiva = más confusión. Diseña múltiples herramientas simples en lugar de una herramienta compleja.

Fuente: "Tool Space Interference in the MCP Era" — Microsoft Research

MCP: ARQUITECTURA Y USO

Arquitectura multi-servidor: Agente de soporte al cliente

Cada servidor corre de forma independiente, gestiona su propio dominio y puede desplegarse o escalarse por separado. El agente no sabe ni le importa dónde viven las herramientas — las descubre todas vía tools/list.

MCP: ARQUITECTURA Y USO

MCP con Claude: Casos de uso prácticos

Con servidores MCP conectados, Claude deja de ser un chatbot y se convierte en un hub de trabajo que accede directamente a tus archivos, APIs, bases de datos y herramientas.

Gestión de código y GitHub

Implementa la funcionalidad descrita en el issue JIRA ENG-4521 y crea un PR en GitHub.

Herramientas: GitHub MCP, Filesystem MCP

Análisis de datos y monitoreo

Revisa Sentry y Statsig para analizar el uso de la feature ENG-4521 y detectar errores.

Herramientas: Sentry MCP, Statsig MCP

Consultas a bases de datos

Encuentra los emails de 10 usuarios aleatorios que usaron la feature ENG-4521 en nuestra base PostgreSQL.

Herramientas: PostgreSQL MCP

Integración de diseño

Actualiza nuestra plantilla de email estándar con los nuevos diseños de Figma publicados en Slack.

Herramientas: Figma MCP, Slack MCP

Automatización de flujos

Crea borradores en Gmail invitando a esos 10 usuarios a una sesión de feedback sobre la nueva feature.

Herramientas: Gmail MCP

Reacción a eventos externos

Claude puede reaccionar a mensajes de Telegram, Discord o eventos webhook en tiempo real mientras trabajas.

Herramientas: Telegram MCP, Discord MCP

Servidores MCP populares: El ecosistema

Más de 890 servidores MCP disponibles. Aquí los más utilizados por categoría.

Archivos y código

Filesystem: lectura/escritura de archivos locales. Git: gestión de repositorios. GitHub: PRs, issues, CI/CD.

Bases de datos

PostgreSQL: consultas SQL. SQLite: bases de datos locales. Memory: grafo de conocimiento persistente.

Productividad y comunicación

Slack: mensajes y canales. Gmail: borradores y envíos. Google Drive: acceso a documentos.

Web y búsqueda

Brave Search: búsqueda web. Fetch: extracción de contenido web. Playwright/Puppeteer: automatización de navegador.

Diseño y APIs

Figma: diseños e interfaces. OpenAPI: conecta cualquier API con spec Swagger. Google Maps: servicios de localización.

Razonamiento avanzado

Sequential Thinking: descomposición de tareas complejas. Context7: documentación actualizada de librerías.

Dónde encontrar servidores MCP:

- Directorio oficial: github.com/modelcontextprotocol/servers
- Directorio comunidad: claudemcp.org (25+ servidores populares)
- Directorio completo: skillsplayground.com/guides/mcp-servers (890+ servidores)
- Registro oficial MCP: mcp.so

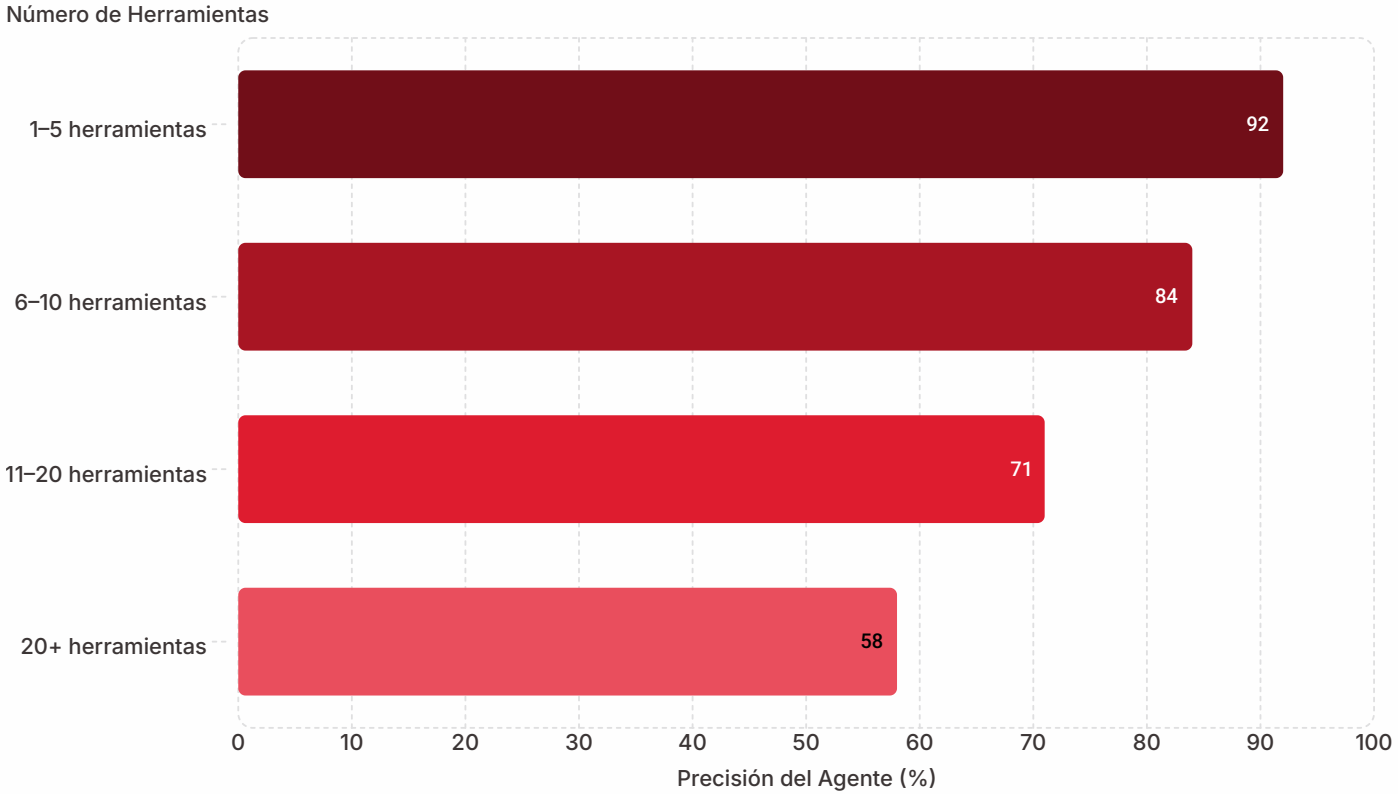
Agenda

1. **RAG: Fundamentos ✓**
   Qué es RAG, por qué lo necesitamos y arquitectura básica.
2. **RAG: Búsqueda y Recuperación ✓**
   Búsqueda vectorial, léxica, semántica e híbrida. Bases de datos y flujo completo.
3. **RAG: Reranking y Precisión**
   Recuperación en dos etapas, cross-encoders, Reciprocal Rank Fusion y optimización de precisión.
4. **RAG: Chunking, Metadatos y Reranking ✓**
   Fragmentación de documentos, metadatos esenciales y recuperación en dos etapas.
5. **RAG: Evaluación, Seguridad y Patrones Avanzados ✓**
   Métricas clave, LLM-as-Judge, riesgos, GraphRAG, Agentic RAG y RAG Multimodal.
6. **MCP: Fundamentos ✓**
   Qué es MCP, historia, protocolo JSON-RPC y comparativa con HTTP/GraphQL.
7. **MCP: Arquitectura y Uso ✓**
   Ciclo de vida, multi-servidor, ecosistema, casos de uso con Claude y cómo conectar servidores.
8. **MCP: Diseño de Herramientas**
   Selección de herramientas, routing jerárquico, grupos por contexto, diferenciación y analytics en producción.

MCP: DISEÑO DE HERRAMIENTAS

El problema de selección de herramientas

La precisión del agente cae drásticamente a medida que aumenta el número de herramientas disponibles. Los LLMs hacen pattern-matching sobre descripciones — los espacios de opciones grandes los abruman.



Con 20+ herramientas en lista plana, el agente selecciona la herramienta incorrecta el 42% de las veces. El diseño de herramientas es tan importante como su implementación.

Source: aitutorial.dev/agents/tool-selection-and-optimization

Solución 1: Routing jerárquico

❌ Anti-patrón: Lista plana (20 herramientas)

The agent sees all 20 tools at once → 58% accuracy.

```
const tools = [
  searchCustomers, searchProducts,
  searchOrders, searchTickets,
  getCustomer, getProduct,
  updateCustomer, updateProduct,
  createOrder, createTicket,
  // ... 10 más
];
// El agente ve 20 herramientas → 58% precisión
```

✅ Solución: Herramienta de routing (1 herramienta)

The agent sees 1 routing tool → 90%+ accuracy.

```
mcp.registerTool("route_to_domain", {
  description: "Enruta al dominio y acción correctos",
  inputSchema: {
    domain: z.enum([
      "customers","products",
      "orders","tickets"
    ]),
    action: z.enum([
      "search","get","update",
      "create","delete"
    ])
  }
});
// El agente ve 1 herramienta → 90%+ precisión
```

En lugar de 20 herramientas planas, el agente recibe 1 herramienta de routing. Elige dominio + acción, y en un segundo paso se invoca la herramienta específica.

Solución 2: Grupos de herramientas por contexto (fase)

Una conversación de soporte tiene fases distintas. Mostrar solo las herramientas relevantes a la fase actual — el agente ve 2-4 en lugar de 20.

01

Fase: Saludo / Autenticación

Herramientas activas (1):

- authenticateCustomer

El agente no necesita ver herramientas de búsqueda ni de tickets todavía.

02

Fase: Diagnóstico

Herramientas activas (2):

- searchKnowledgeBase
- checkSystemStatus

El agente se enfoca en encontrar la causa del problema.

03

Fase: Resolución

Herramientas activas (2):

- createTicket
- scheduleCallback

El agente no necesita herramientas de búsqueda en esta fase.

❌ Sin grupos de contexto

20 herramientas visibles → 58% precisión

✅ Con grupos de contexto

2-3 herramientas por fase → 90%+ precisión

El principio: el agente no necesita ver todas las herramientas todo el tiempo. Filtra por fase o contexto para maximizar la precisión de selección.

Solución 3: Diferenciación clara entre herramientas similares

Cuando dos herramientas suenan igual, el LLM las elige al azar o llama a ambas. La solución: cada herramienta necesita un propósito distinto con guía explícita de "Usar cuando" / "NO usar cuando".

`search_products_by_text`

Descripción: Búsqueda de texto completo.

✅ Usar cuando: el cliente describe el producto con palabras ("ratón inalámbrico").

❌ NO usar cuando: tienes el SKU exacto.

Parámetro: query: string

`get_product_by_sku`

Descripción: Búsqueda exacta por SKU.

✅ Usar cuando: el cliente proporciona un SKU ("PROD-001").

❌ NO usar para búsquedas de texto libre.

Parámetro: sku: string

`filter_products_by_attributes`

Descripción: Filtro estructurado por atributos.

✅ Usar cuando: el cliente especifica categoría, precio o marca.

❌ NO usar para búsqueda de texto.

Parámetros: category?: string, priceMax?: number

El agente ahora elige correctamente:

- "ratón inalámbrico" → search_products_by_text ✓
- "PROD-001" → get_product_by_sku ✓
- "ratones por menos de $20" → filter_products_by_attributes ✓

MCP: DISEÑO DE HERRAMIENTAS

Analytics de herramientas: Optimización continua en producción

En producción, monitorear cada llamada a herramientas permite detectar problemas sistémicos y optimizar el rendimiento del agente.

Herramientas sin uso

Detectar herramientas que nunca se invocan.

Acción: Eliminar o consolidar.

Ejemplo de alerta: "Eliminar herramienta sin uso: legacy_search"

Herramientas con alta tasa de error

Detectar herramientas que fallan frecuentemente.

Acción: Revisar manejo de errores o mejorar descripción.

Ejemplo de alerta: "flaky_tool falla el 40% — revisar error handling"

Herramientas lentas

Detectar herramientas con alta latencia promedio.

Acción: Implementar caché o optimizar la API.

Ejemplo de alerta: "slow_api promedia 3000ms — considerar caché"

Key metrics to track:

- call_count: número de invocaciones
- success_rate: tasa de éxito (%)
- avg_latency_ms: latencia promedio en ms
- last_used: última vez utilizada

Las herramientas que el agente nunca usa o usa mal son señales de diseño deficiente — no de capacidad del modelo. Itera sobre descripciones y esquemas antes de cambiar el LLM.